Report of mini project step 2 of mandelbrot.

Mandelbrot set problem (same as in first miniproject):

The mandelbrot set is a 2 dimensional set in the complex plane. The mandlebrot set is an uncountable set of values of c in the complex plane.
The mandelbrot can be partially calculated with a fixed resolution in a computational way. The algorithm is as follows:

Define fixed region dimensions (x,y)
Generate a grid of complex numbers over the fixed defined region with evenly spread x and y values.
For each point generate value: x+iy as c
For each point now:
    Initialize z = 0
    For n to max iterations (max iteration should be a fixed number)
        Compute: zn+1 = zn 2 + c
        If absolute value of zn+1 > 2: store zn+1 in given point and move to next point

        Else:
        If loop terminates before zn+1 > 2. Store max iterations' value

When loop is finished you have obtained the mandelbrot set in the form of a matrix with iteration counts

The problem is an embarrassingly parallel problem as it is highly paralizable since each value of each grid location can be calculated individually and does not rely on any other data than the complex grid generated equally for all locations.

The problem has many options for implementations. Some will perform better than others in terms of time to execute per loop.

Implementations:

For the second step of the mini project, these given implementations of computing the mandelbrot set has been introduced and performed:

1. Parallel processing
2. Dask locally
3. Dask on cluster

The exact implementation is included in the code in the form of a jupyter notebook, as well as their performance benchmarks.

1. Parallel processing:

The parallel processing method defines a worker function that works from a given start row to a given end row. The parallel function takes an argument which indicates the number of threads which should run parallel.

The function then divides the total rows equally into the threads and the worker nodes start processing. Each worker processes its assigned rows using a vectorized approach on its assigned piece of the grid.

2. Dask

The Dask solution uses a Dask array and lazy operations to calculate the pixels of the mandelbrot set. Like the Vectorized solution the Dask function makes use of an mask array of boolean values indicating if a cell has escaped or not. It constructs a lazy computation graph where operations are applied chunk-wise across the array.

3. Dask on cluster

The Dask on cluster works almost as the Dask locally. It diverges in the sense that it needs to connect to a Dask scheduler server node and it makes use of a fixed chunk size of 128 elements to control task granularity and improve scheduling efficiency. It also makes use of the distributed lib from Dask and uses

“persist” and “wait” functions to materialize intermediate results, reduce task graph size, and ensure stable execution across workers.

Benchmarking methodology:

The benchmarking utilized timeit.repeat function. This repeat generate a list of runs which the average can be derived from.

To measure the performance when scaling the problem size. The given resolutions of the grid has been tested with each implemented solution:

1024x1024
2048x2048
4096x4096
8192x8192

Each test ran on a local laptop desktop.

No cloud machine was used for the benchmark of all tests except the dask cluster test.

PC specs:

=== Static System Information ===

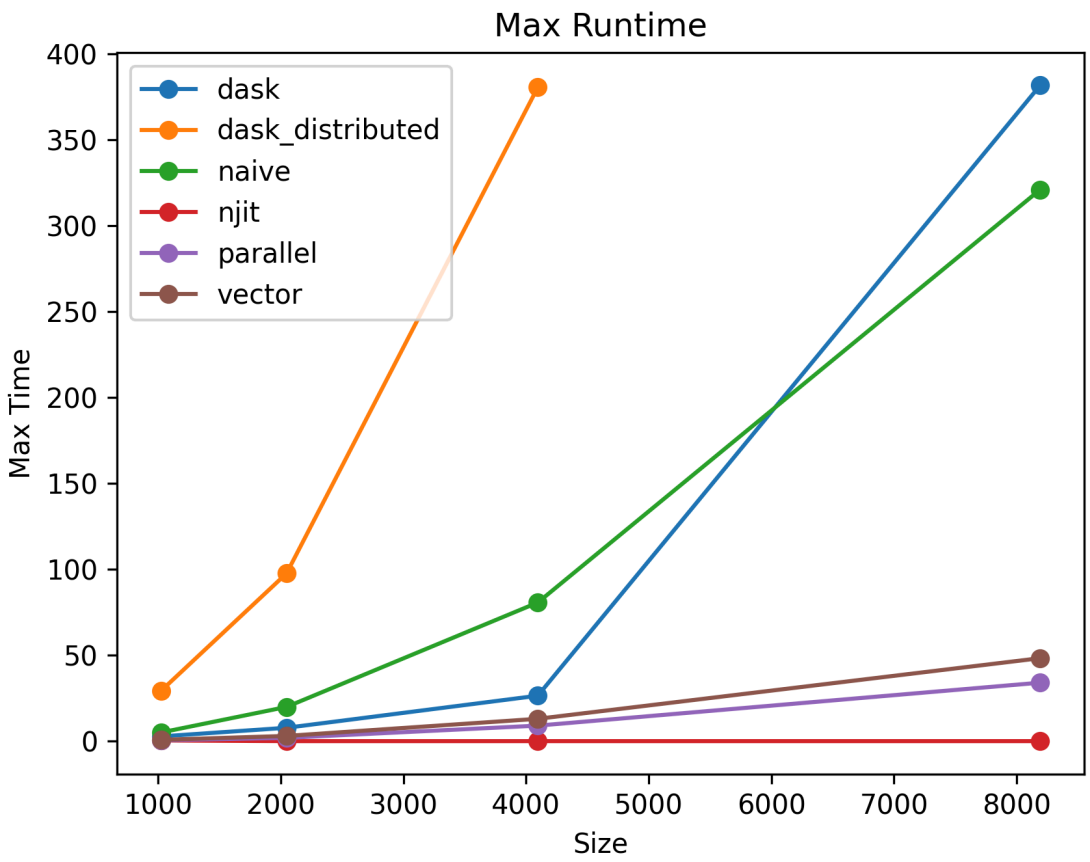
System: Windows 10
Machine: x86_64
Processor: x86_64
CPU Brand: AMD ruzen 9 5950x 16 core processor
CPU Arch: X86_64
CPU Bits: 64
Number of Physical Cores: 16
Number of Logical Cores: 32

=== CPU Cache Sizes ===
L1 Data Cache: 16x32 KBytes
L1 Instruction Cache: 16x32 KBytes
L2 Cache: 16x512 KBytes
L3 Cache: 2x32 MBytes

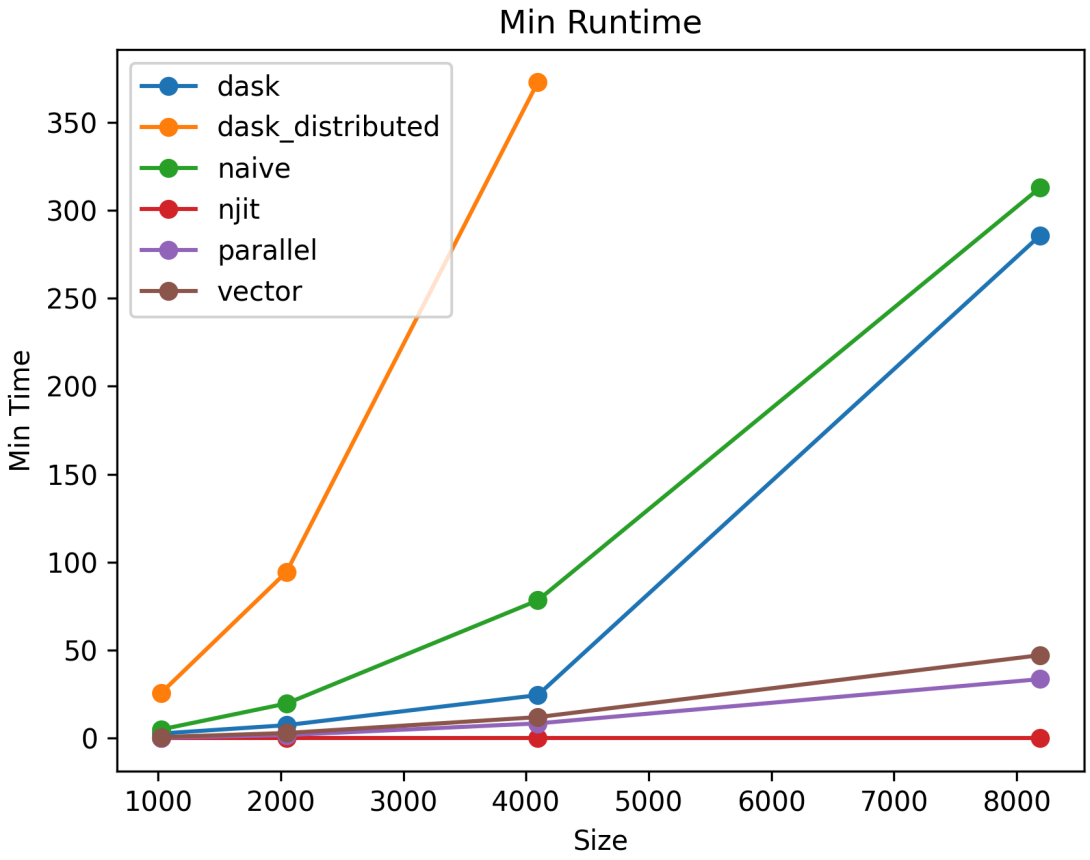
=== Memory ===
Total RAM: 32 GB

Benchmarking results

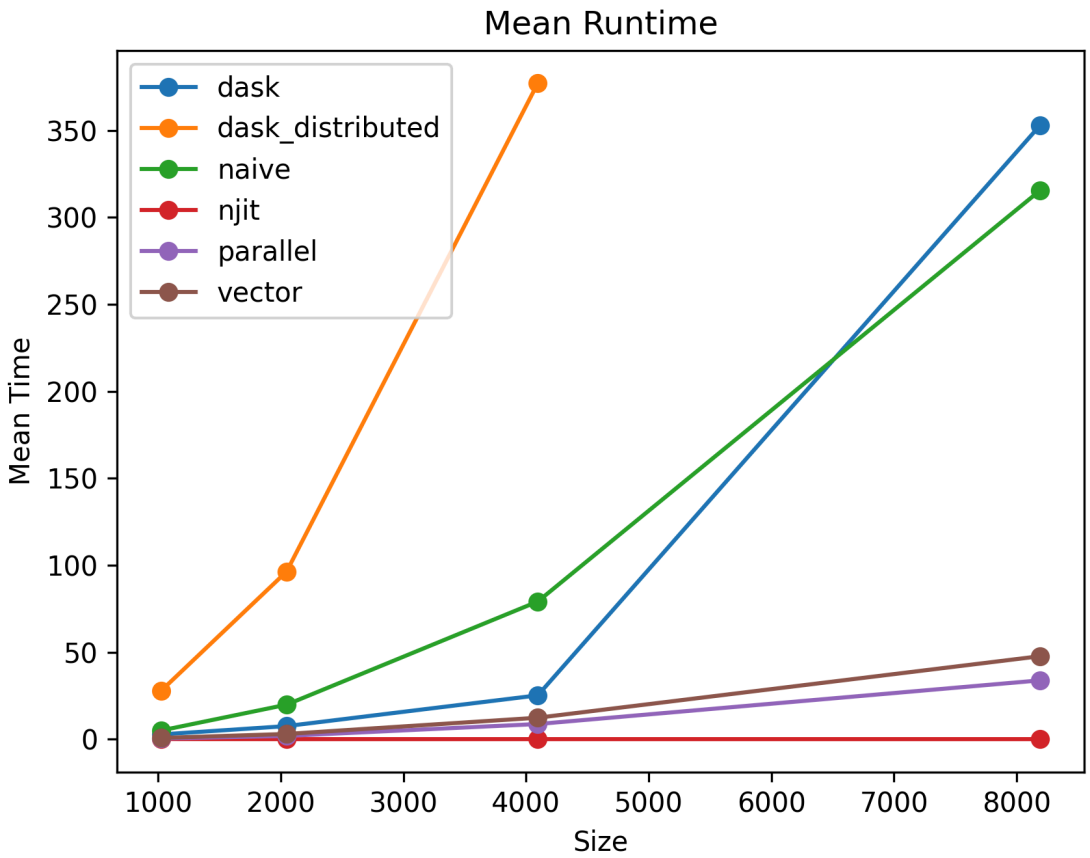
Max time result across all tests:



Min result across all tests:



Min result across all tests:



Evaluation of benchmarking results.

Parallel seems to be the best of the three new implementations for efficiently calculating the Mandelbrot set in this setup. By splitting the work across threads and letting each worker operate on a vectorized block of data, it achieves good performance for these problem sizes. However, the scaling behavior remains similar to the vectorized approach, with improvements mainly coming from parallel execution rather than reduced complexity.

Dask performs better than the naive implementation for smaller problem sizes, but it scales poorly compared to vectorized and parallel approaches. This indicates that the overhead from task scheduling and chunk management outweighs its benefits for this type of workload. Without sufficiently large datasets or a well-optimized chunking strategy, Dask introduces too much overhead to be efficient in this scenario.

The Dask distributed implementation performs significantly worse than all other methods tested. It shows high initial overhead and poor scaling as the problem size increases. This is likely due to communication overhead, task scheduling costs, and limited resources on the cluster nodes. For a problem of this size, distributed execution is not suitable, and the overhead dominates any potential parallel gains. The instability at larger sizes (e.g., 8192×8192) further highlights that this approach is not practical in this context.